

Implementing Tests for LLM Prompts with Semantic Assertions

Md Abdulla AL Mahamud Rosi
[abdulla.rosi@stud.fra-uas.de]

Abstract—Traditional software testing relies on exact string matching to verify program output correctness, a strategy that is fundamentally incompatible with Large Language Model (LLM) outputs. LLMs are non-deterministic: the same factually correct response may take many valid surface forms. This paper presents the LLM Semantic Evaluator, a .NET/C# framework built on Microsoft.Extensions.AI that replaces lexical matching with two independent semantic validation strategies. The first strategy converts both the expected and actual LLM output to high-dimensional embedding vectors and measures cosine similarity. The second employs a secondary LLM as a judge, using a chain-of-thought prompt that requires the judge to reason step by step before assigning a numeric score—an approach directly inspired by the G-Eval evaluation methodology. Each test case is executed three times and is marked as passed if at least two runs pass. Configuration is loaded from appsettings.json and bound to a typed class via the standard IOptions<T> pattern; all services are registered in the dependency injection container and receive their dependencies through constructor injection. The framework supports OpenAI GPT, and locally hosted Ollama models through a unified JSON configuration file with no code changes required when switching providers. Reports are generated in four formats: plain text, JSON, CSV, and an interactive HTML dashboard. Empirical evaluation on a 138-case dataset spanning six knowledge domains yielded a 99.3% pass rate with OpenAI gpt-5-mini and 82% with Ollama llama3.2:3b across a 50-case subset. The single OpenAI failure was caused by a mismatch between the reference expected output and a historically more accurate model response. The Ollama performance gap is attributable to hardware constraints that require a smaller model and a reduced dataset, and to the model's limited ability to apply a structured scoring rubric reliably.

Keywords—semantic similarity, cosine similarity, LLM evaluation, embeddings, G-Eval, chain-of-thought, .NET, C#, non-deterministic testing, Ollama, OpenAI, LLM-as-a-Judge, Microsoft.Extensions.AI, dependency injection

I. INTRODUCTION

Automated regression testing is a fundamental practice in software engineering. For deterministic systems, correctness can be asserted with simple string equality. For Large Language Models, this approach fails entirely: an LLM may produce dozens of semantically equivalent but

lexically distinct responses to the same prompt. Testing frameworks designed for traditional software cannot handle this variability without producing large numbers of false failures.

The non-determinism problem is aggravated by the rapid integration of LLMs into production systems. Teams building question-answering modules, summarisation services, code generators, and conversational agents all need regression tests that verify semantic correctness regardless of phrasing. Standard metrics such as BLEU and ROUGE penalise valid paraphrases, making them equally unsuitable.

This paper presents the LLM Semantic Evaluator, a .NET/C# framework built to satisfy the requirements of ML project 25/26-08. The framework implements both required semantic validation approaches—embedding-based cosine similarity and LLM-as-a-Judge with chain-of-thought scoring—and combines them with a majority-vote strategy (repeated test execution and pass decision) across repeated runs. It is built on Microsoft.Extensions.Hosting and Microsoft.Extensions.AI, using standard dependency injection and configuration binding patterns. It supports OpenAI, and local Ollama models, loads test cases from JSON, and generates reports in four formats including an interactive HTML dashboard.

The goal of the system is to address a practical software engineering problem: given an LLM-powered application, how can a development team run automated tests that verify the application gives semantically correct answers, even though the exact wording of each answer differs from run to run? The framework treats this as an engineering problem by combining two complementary validators, running each test case multiple times, and requiring a majority of runs to pass before marking a test as failed. The result is a stable, repeatable test suite that tolerates normal LLM variability while reliably detecting genuinely wrong answers.

The remainder of this paper is organised as follows. Section II surveys relevant background. Section III describes the methodology and system design. Section IV details each component's implementation. Section V presents experimental results with analysis. Section VI covers unit testing. Section VII discusses key findings, and Section VIII concludes.

II. LITERATURE REVIEW

Text representation has evolved from count-based methods such as Bag of Words and TF-IDF [1], which capture word frequency but miss synonymy and paraphrase, to static dense embeddings such as Word2Vec and GloVe

[2] that map tokens into a shared semantic space. Transformer-based models such as BERT and OpenAI’s embedding series [3] produce contextualised representations that adapt to surrounding text, enabling meaningful similarity comparisons even when surface forms differ substantially.

Cosine similarity is the standard metric for comparing embedding vectors because it measures directional alignment independently of vector magnitude [4]. For unit-normalised embeddings such as those produced by OpenAI’s text-embedding-3 series, cosine similarity and dot product yield identical results, confirming cosine similarity as a robust choice for this task.

The G-Eval framework [5] demonstrated that prompting a secondary LLM to reason step by step before assigning a numeric evaluation score—a chain-of-thought approach—produces judgements more strongly correlated with human opinion than n-gram metrics. Requiring explicit reasoning forces the judge to analyse the response rather than pattern-match to a number. This methodology directly informs the LLM judge component implemented in this project: the judge prompt requires multi-sentence reasoning before the score is written, and the reasoning text is preserved in the reports for auditability.

Multi-run aggregation and majority voting are established techniques for managing non-determinism in probabilistic systems. Running the same query multiple times and aggregating results reduces the probability that a single unusual model output causes a test to fail incorrectly [6].

III. METHODOLOGY

A. Problem Formulation

The framework addresses a concrete software engineering problem: given an LLM that is expected to answer questions correctly, how does a development team run automated regression tests that detect wrong answers without failing whenever the model rephrases a correct response? Exact string matching cannot work because LLM responses are naturally variable. The framework therefore defines a test case as passing if the LLM’s actual response carries the same semantic content as the reference expected output, regardless of phrasing.

Formally, given a test case comprising a natural language prompt prompt, a reference expected output expectedOutput, and optional evaluation criteria evaluationCriteria, the goal is to determine whether the actual LLM response actualResponse correctly answers prompt with the same semantic content as expectedOutput. A single test run passes if either:

```
(1) CosineSimilarity(Embed(expectedOutput),
Embed(actualResponse)) >=
EmbeddingThreshold, or
```

```
(2) GetJudgeScore(prompt, expectedOutput,
actualResponse, evaluationCriteria) >=
JudgeThreshold.
```

The overall test passes if at least MinimumPassingRuns out of NumberOfRuns repetitions satisfy the above condition (majority vote). This design means a single network error or an unusually phrased response does not immediately fail a test case.

B. Why This Approach is Needed

The OR combination of both validators addresses a structural limitation of each method individually. Embedding similarity is fast and consistent but produces low scores when the expected output is a short phrase and the actual response is a full sentence carrying the same fact. For example, if the expected output is “Paris” and the model responds, “The capital of France is Paris”, the cosine similarity between the two embeddings is approximately 0.45—well below the 0.85 threshold—even though the answer is correct. The LLM judge compensates by reasoning about semantic content directly.

Conversely, the embedding validator provides a reliable fast path for clear semantic matches and does not depend on the judge model’s ability to follow a scoring rubric. When the judge model is too small to apply the rubric consistently, the embedding path can still pass correct answers if the similarity is above threshold.

The majority-vote strategy across three runs addresses the non-determinism of LLM outputs. Even at temperature 0.0, LLMs can occasionally produce an atypical response. Requiring two out of three runs to pass means a single outlier does not cause a false failure.

C. On Pass Rates Below 100%

A pass rate below 100% in semantic evaluation does not necessarily mean the LLM under test is producing wrong answers. There are two distinct explanations for failures, and distinguishing them is important for correctly interpreting results.

The first explanation is a test dataset quality issue: the reference expected output is less accurate than the model’s actual response. Both validators fail because the response genuinely disagrees with the reference, even though the model is right. This is a desirable property of the framework—it surfaces disagreement so a developer can inspect the case and decide whether to correct the expected output or the model’s behaviour.

The second explanation is judge model miscalibration: the judge LLM assigns low scores to factually correct, well-formed responses. This is a model capability problem rather than a test dataset problem. It is more likely with small models that lack the instruction-following stability needed to apply a structured scoring rubric consistently. The framework makes this diagnosable through the per-run judge reasoning text stored in the JSON report.

For the Ollama run specifically, a third factor applies: hardware constraints on the evaluation machine required using llama3.2:3b and limiting the dataset to 50 test cases,

since larger models and the full dataset caused HTTP requests to exceed the configured 100-second timeout. The 82% pass rate therefore reflects both the model's calibration limitations and the hardware-imposed constraints and is not directly comparable to the OpenAI run on 138 cases.

D. System Architecture

The application is structured as a Microsoft.Extensions.Hosting generic host. EvaluatorService extends BackgroundService, the standard pattern for running a background task within the hosting infrastructure. All services are registered in the dependency injection container in Program.cs and receive their dependencies through constructor injection. No class instantiates another class with new. No class reads appsettings.json directly.

The evaluation flow is sequential: EvaluatorService calls ITestLoader.LoadTestsAsync() to load the test cases, passes the resulting list to TestRunner.RunAllAsync(), and then passes the results list to ReportGenerator.GenerateAsync(). Each service has a single responsibility and is independently testable through its interface. Fig. 1 shows the component diagram of the system.

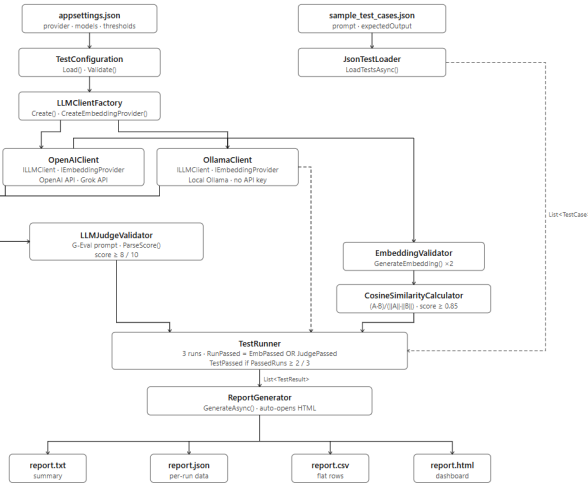


Fig. 1: Component diagram of the LLM Semantic Evaluator

E. Test Case Structure

Test cases are stored in a JSON file (default: data/sample_test_cases.json). Each object in the array contains the fields defined in Table I. The loader accepts both a bare JSON array and a {"tests": [...]} wrapped format. The category field is used for report grouping; evaluation_criteria provides task-specific scoring guidance to the judge prompt. Both are optional.

TABLE I. TEST CASE JSON SCHEMA

Field	Required	Description
Id	Yes	Unique identifier (e.g. factual_001)
Prompt	Yes	Query sent verbatim to the LLM under test

expected_output	Yes	Reference answer used by both validators
Category	No	Label for report breakdown. Default: general
evaluation_criteria	No	Criteria injected into judge prompt

A representative entry from sample_test_cases.json illustrates the schema:

```

{
  "id": "factual_009",
  "category": "factual",
  "prompt": "What is the smallest country in the world?",
  "expected_output": "Vatican City",
  "evaluation_criteria": "Response must identify Vatican City"
}

```

F. Repeated Test Execution and Pass Decision

Each test case is executed three times (NumberOfRuns = 3) with temperature set to 0.0 to minimise sampling variance between runs. After all three runs complete, the framework counts how many runs passed (PassedRunsCount). A run is considered passed if either the embedding validator or the judge validator returns a score at or above its respective threshold. The overall test case is marked as **Passed** if PassedRunsCount ≥ MinimumPassingRuns (default: 2), and **Failed** otherwise.

This means one failed run — caused by an API timeout, a rate-limit error, or a single atypical model response — does not fail the entire test case. Two independent failures on the same test case, however, are treated as a genuine signal that the model's answer is wrong.

IV. IMPLEMENTATION

A. Technology Stack and Framework Conventions

The project is implemented in C# targeting .NET 10. It uses the following standard framework packages: Microsoft.Extensions.Hosting for the application lifecycle and hosted service infrastructure; Microsoft.Extensions.DependencyInjection for the DI container; Microsoft.Extensions.Configuration.Json for configuration loading; Microsoft.Extensions.AI for the IChatClient and IEmbeddingGenerator<string, Embedding<float>> standard interfaces; Microsoft.Extensions.AI.OpenAI for the OpenAI client adapters; and OllamaSharp for the Ollama client, which implements the same MS AI interfaces natively. All runtime output is written through ILogger<T> injected via the DI container rather than Console.WriteLine, so the application works correctly in any hosting environment including Docker containers and CI pipelines where a console may not be available. Each C# class lives in its own dedicated file; no file defines more than one public type.

Using the framework interfaces rather than writing custom HTTP clients is a deliberate engineering decision. It

means that all services depending on `IChatClient` or `IEmbeddingGenerator` are not coupled to any specific provider. Switching from OpenAI or Ollama requires only a change to `appsettings.json`. This is the benefit that justifies adopting the `Microsoft.Extensions.AI` framework: it delivers provider-agnostic interfaces that the application does not need to implement or maintain.

B. Configuration and Dependency Injection

`Program.cs` uses `Host.CreateDefaultBuilder()` as the application entry point. Configuration is bound to `TestConfiguration` via `services.Configure<TestConfiguration>(context.Configuration)`; services receive `IOptions<TestConfiguration>` through constructor injection and read `.Value`. No service reads the JSON file directly. All services are registered in the DI container as singletons. `IChatClient` and `IEmbeddingGenerator<string, Embedding<float>>` are registered by delegating to `LLMClientFactory`. `EvaluatorService` is registered as a hosted service via `AddHostedService<EvaluatorService>()`.

C. LLMClientFactory and Multi-Provider Support

`LLMClientFactory` implements the factory pattern using the `Microsoft.Extensions.AI` framework. It creates the correct `IChatClient` and `IEmbeddingGenerator` based on the `Provider` and `EmbeddingProvider` fields in `appsettings.json`. For OpenAI and , it uses `OpenAIClient.GetChatClient().AsIChatClient()` and `OpenAIClient.GetEmbeddingClient().AsIEmbeddingGenerator()` from the official `Microsoft.Extensions.AI.OpenAI` package. For Ollama, it uses `OllamaApiClient`, which implements both interfaces natively.

The base URLs for OpenAI is defined as private constants inside `LLMClientFactory` and nowhere else. The `OllamaBaseUrl` is read from `appsettings.json` because users may run Ollama on a non-default host or port. To add a new provider, a developer adds its settings to `appsettings.json` and `TestConfiguration` and adds one case to each switch statement in `LLMClientFactory`. No other file needs to change.

D. EmbeddingValidator

`EmbeddingValidator.ValidateAsync()` calls the `IEmbeddingGenerator` twice—once for the expected output and once for the actual response—then passes both `float[]` vectors to `CosineSimilarityCalculator`. An empty or null actual response is treated as an automatic failure without calling the embedding API, avoiding unnecessary charges. Any API exception is caught and returned as a failed `ValidationResult` with the error message stored in the `Reasoning` field, so a single network error does not abort the whole test run.

E. CosineSimilarityCalculator

The calculator computes the cosine similarity formula in a single pass through both vectors, accumulating the dot product and both squared magnitudes simultaneously:

```
dotProduct += a[i] * b[i];
magnitudeA += a[i] * a[i];
magnitudeB += b[i] * b[i];
```

```
similarity = dotProduct / (sqrt(magA) *
sqrt(magB));
```

The result is clamped to `[-1, 1]` to absorb floating-point precision artefacts. A zero-magnitude vector returns 0.0 rather than throwing a division-by-zero exception, as specified by the interface contract.

F. LLMJudgeValidator and the G-Eval Chain-of-Thought Approach

G-Eval [5] is an NLG evaluation methodology in which a secondary LLM acts as a judge. Its defining characteristic is that the judge is required to reason step by step before assigning a score. This chain-of-thought requirement produces scores more strongly correlated with human judgement than prompts that request a score directly, because it forces the model to analyse the content rather than pattern-match to a number.

This project implements the G-Eval approach in `LLMJudgeValidator`. The `BuildJudgePrompt()` method constructs a structured prompt that is sent to the configured `IChatClient` for every test run. The prompt has two explicit steps: in Step 1 the judge must reason in two to four sentences about whether the actual answer addresses the query, whether it matches the expected answer in meaning, and whether there are any factual errors or omissions. Only after completing that reasoning is the judge asked, in Step 2, to assign a numeric score on a defined 1–10 scale and write it on the final line in the format `SCORE: N`. The prompt used by `BuildJudgePrompt()` is shown below:

```
You are an expert evaluator assessing AI
response quality.
Query:      {prompt}
Expected:   {expectedOutput}
Actual:     {actualOutput}
{criteriaSection}
Step 1 - Reason step by step (2-4
sentences):
    - Does the actual answer address the
      query?
    - Does it match the expected answer in
      meaning?
    - Any factual errors or key omissions?

Step 2 - Scoring scale:
    10 = Semantically identical to
    expected
    8-9 = Same core meaning, minor
    differences
    6-7 = Mostly correct but incomplete
    4-5 = Partially correct, missing key
    points
    1-3 = Wrong, irrelevant, or misleading

Write reasoning first, then ONLY: SCORE:
<n>
```

The reasoning and score are extracted separately from the judge's free-form response. The primary extraction pattern matches `SCORE: N` case-insensitively; a fallback pattern finds the first standalone integer in `[1, 10]` to handle responses that do not follow the required format. The

extracted reasoning text is stored in the JSON and HTML reports alongside every test run, making every judge decision fully auditable. The optional evaluation_criteria field from the test case is injected when present.

An important implementation detail: gpt-5-mini and OpenAI reasoning models (o1, o3) reject the temperature parameter with HTTP 400. LLMJudgeValidator detects this via the SupportsTemperature() method and omits the parameter for those models, allowing the provider to use its own default.

G. TestRunner

TestRunner.RunAllAsync() iterates over all test cases and calls RunSingleTestAsync() for each. Within a single test, it loops NumberOfRuns times. Each iteration sends the prompt to the IChatClient, runs both validators sequentially, and constructs a TestRun record with all scores and flags. A configurable Task.Delay between runs avoids hitting API rate limits. Any exception during a run is caught and recorded as a failed run; the loop continues to the next run. After all runs, the TestResult is aggregated: PassedRunsCount counts runs where RunPassed (= EmbeddingPassed OR JudgePassed) is true, average scores are computed, and the overall Passed flag is set by the majority-vote comparison.

H. ReportGenerator

ReportGenerator.GenerateAsync() creates a reports/ directory and writes four output files. The plain text report (report.txt) provides a human-readable summary with per-test details. The JSON report (report.json) serialises the full structured results including per-run judge reasoning text, enabling detailed post-hoc analysis. The CSV report (report.csv) produces one flat row per test case, which can be directly opened in any spreadsheet application for sorting, filtering, and charting without additional tooling. The HTML report (report.html) uses a self-contained template with embedded JavaScript that renders charts, per-category breakdowns, and expandable per-test detail rows. On completion it is opened automatically in the system default browser.

V. RESULTS

A. Experimental Setup

Two evaluation runs were performed. The primary run used sample_test_cases.json, a 138-case dataset spanning six knowledge domains (factual, definitions, history, science, math, reasoning), with OpenAI gpt-5-mini as the chat model and text-embedding-3-small for embeddings. The second run used quick_tests.json, a 50-case dataset covering the same six domains, with Ollama llama3.2:3b as the chat model and nomic-embed-text for embeddings. The Ollama dataset was deliberately limited to 50 cases because each run on llama3.2:3b required significantly more execution time than the OpenAI run; using the full 138-case dataset caused individual HTTP requests to exceed the configured 100-second timeout on the evaluation machine. Each test case was executed three times with majority-vote

threshold of 2/3 runs. Table II shows the provider configuration for each run.

TABLE II. EXPERIMENTAL CONFIGURATIONS

Parameter	OpenAI Run	Ollama Run
Chat model	gpt-5-mini	llama3.2:1b
Embedding model	text-embedding-3-small	nomic-embed-text
Embedding dims	1,536	768
Temperature	model default (gpt-5-mini)	0.0
Runs / MinPass	3 / 2	3 / 2
EmbeddingThreshold	0.85	0.85
JudgeThreshold	8	8

B. Console Output During Execution

During execution, the console logs each test in real time, printing the test index, ID, and after completion the pass/fail verdict plus how many of the three runs passed. Fig. 2 shows a representative console screenshot from the OpenAI run.

```

info: LLMSemanticEvaluator.Services.EvaluatorService[0]
info: LLMSemanticEvaluator.Services.EvaluatorService[0]
      Embedding provider : openai
info: LLMSemanticEvaluator.Services.EvaluatorService[0]
      Chat model       : gpt-5-mini
info: LLMSemanticEvaluator.Services.EvaluatorService[0]
      Embedding model  : text-embedding-3-small
info: LLMSemanticEvaluator.Services.EvaluatorService[0]
      Runs per test    : 3
info: LLMSemanticEvaluator.Services.EvaluatorService[0]
      Min passing runs : 2
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: G:\FUAS\Ros\llm-semantic-evaluator\LLMSemanticEvaluator
info: LLMSemanticEvaluator.Services.EvaluatorService[0]
      Embedding threshold : 0.85
info: LLMSemanticEvaluator.Services.EvaluatorService[0]
      Judge threshold     : 8/10
info: LLMSemanticEvaluator.Services.EvaluatorService[0]
info: LLMSemanticEvaluator.Services.EvaluatorService[0]
      Loading test cases...
info: LLMSemanticEvaluator.Services.EvaluatorService[0]
      Loaded 50 test cases.
info: LLMSemanticEvaluator.TestRunner[0]
      Starting: 50 tests x 3 runs each
info: LLMSemanticEvaluator.TestRunner[0]
      [1/50] factual_022 ~ PASS (3/3 runs passed)

```

Fig. 2: Console output during OpenAI run showing real-time test progress and results

C. Evaluation Reports as Deliverables

The four report formats serve different purposes. The HTML report (report.html) is the primary interactive deliverable: it opens in any web browser and presents the overall pass rate, a per-category bar chart, and a per-test table where each row is colour-coded by pass/fail status. Expanding a row reveals the model's actual response and the full judge reasoning text for each of the three runs, making individual failures immediately diagnosable without opening any other file.

The CSV report (report.csv) is designed for quantitative analysis. It provides one flat row per test case with columns for test ID, category, pass/fail verdict, passed run count, average embedding score, and average judge score. The file can be opened directly in Excel, LibreOffice Calc, or any data analysis tool without any conversion step,

enabling sorting by score, filtering by category, or charting distributions.

The JSON report (report.json) is the most information-dense output. For each test case it records the prompt, expected output, overall verdict, average scores, and a full array of per-run entries—each containing the model’s response, the embedding score, the judge score, and the judge’s complete reasoning text. The plain text report (report.txt) provides a quick summary for console-level review.

D. Overall Results

The primary evaluation used a dataset of 138 test case from 6 categories (factual, definitions, history, science, math, reasoning), with gpt-5-mini as the chat model and text-embedding-3-small for embeddings. 137 test cases passed, yielding a 99.3% pass rate. Five of the six categories achieve 100%; the single failure occurs in the history category and is discussed in Section V.F. The average embedding score of 0.52 is well below the 0.85 threshold while average judge scores 10 out of 10. Which confirms that the OR logic between the two validators is essential to the framework’s correctness: nearly all passes in this run were driven by the judge path rather than the embedding path as shown in Fig 3. The embedding score is expected to fall below threshold when the reference expected output is a short phrase such as "Paris" or "4" while the model’s response is a full explanatory sentence. The judge path compensates in all but one case. Fig. 4 shows the HTML dashboard for the OpenAI run.

TEST RESULTS — EMBEDDING SCORE · JUDGE SCORE · PER RUN

Test ID	Category	Embedding score / run			Judge score / run			Runs passed	Status
		R1	R2	R3	R1	R2	R3		
factual_021	factual	0.45	0.45	0.45	10/10	10/10	10/10	3/3	pass
factual_022	factual	0.50	0.52	0.49	10/10	10/10	10/10	3/3	pass
factual_023	factual	0.31	0.31	0.31	10/10	10/10	10/10	3/3	pass

Fig. 3: Test passes via judge path despite low embedding score

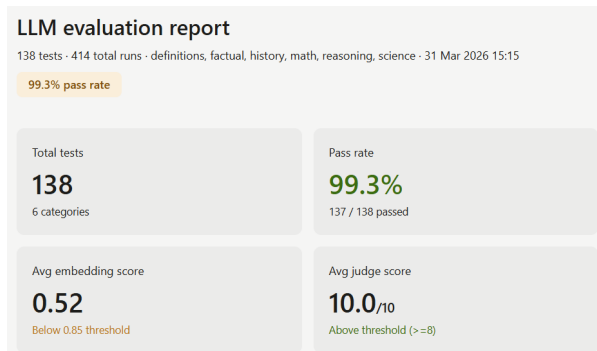


Fig. 4: HTML report dashboard — OpenAI run (report.html)

The second evaluation used a dataset of 50 test case covering the same six domains, with llama3.2:3b as the chat model and nomic-embed-text for embeddings. 41 of 50 test cases passed, yielding an 82% pass rate with an average judge score of 7.8/10. The dataset was limited to 50 cases because running llama3.2:3b locally on the evaluation machine was significantly slower than the OpenAI run; using the full 138-case dataset caused individual HTTP

requests to exceed the configured 100-second timeout, which is a hardware constraint of the local machine rather than a limitation of the framework itself. The 82% pass rate therefore reflects two compounding factors: the hardware-imposed requirement to use a smaller model (llama3.2:3b), and that model’s reduced ability to apply a structured scoring rubric consistently. Fig. 5 shows the Ollama run HTML dashboard where the lower average judge score is clearly flagged.

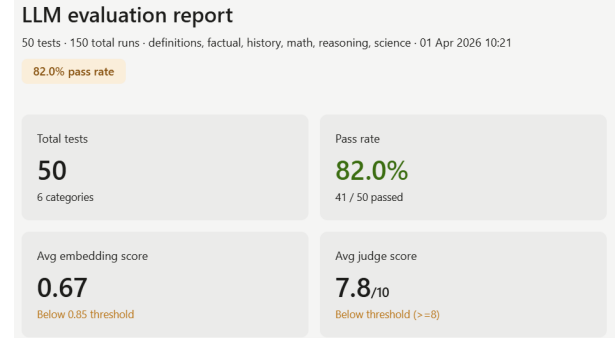


Fig. 5: HTML report dashboard — Ollama run (report.html)

To isolate provider behaviour from dataset size effects, both runs were also compared on the same 50-case subset under identical configuration (EmbeddingThreshold 0.85, JudgeThreshold 8). Table III summarises the results of this controlled comparison.

TABLE III. OVERALL RESULTS — OPENAI VS OLLAMA

Metric	OpenAI	Ollama
Total tests	50	50
Passed	50	41
Failed	0	9
Pass rate	100%	82%
Avg embedding score	0.57	0.67
Avg judge score	10/ 10	7.8 / 10

On the same 50 test cases, OpenAI passes all tests while Ollama fails 9. The average judge score difference (10/10 vs 7.8/10) is the primary driver of this gap: gpt-5-mini applies the G-Eval scoring rubric consistently across all six domains, while llama3.2:3b produces inconsistent scores that fall below the JudgeThreshold. Notably, Ollama’s average embedding score (0.67) is slightly higher than OpenAI’s (0.57), indicating that the embedding validator behaves comparably across both providers — the performance difference is isolated to the judge path.

E. Report Outputs

The HTML report opens in any web browser without additional tooling. The dashboard header shows the overall pass rate, average embedding score, average judge score,

and provider configuration at a glance. Below the header, a bar chart breaks results down by category so failing categories are immediately visible without reading individual test entries. The per-test table lists every test case with pass/fail status, passed run count, and average scores. Expanding a row reveals each individual run: the model’s actual response text, the embedding score, the judge score, and the judge’s full chain-of-thought reasoning. This makes it possible to diagnose any failure without examining raw JSON. For example, a developer can immediately see whether a failure was caused by a genuinely wrong model answer, a judge calibration problem, or a mismatch between the expected output and what the model considers correct.

The JSON report records the complete per-run data for every test case: prompt, expected output, overall verdict, average scores, and a full array of per-run entries each containing the model’s response, the embedding score, the judge score, and the judge’s full reasoning text. Fig. 7 shows a case where the embedding score (0.45) is well below the 0.85 threshold, but the judge correctly scores the response 10/10, causing the run to pass via the judge path. This excerpt illustrates concretely why the OR logic is necessary: a pure embedding approach would have incorrectly failed this test. The CSV report provides one flat row per test case with columns TestId, Category, Passed, PassedRuns, TotalRuns, AverageEmbeddingScore, and AverageJudgeScore. This format opens directly in Excel or LibreOffice Calc without conversion, enabling sorting by score, filtering by category, or generating custom charts.

F. OpenAI Failure Analysis

One test case failed in the OpenAI run. Test case history_006 asks: “Which empire was Julius Caesar a part of?” The expected output is “The Roman Empire”. The model responded that Caesar was a leading general and statesman of the late Roman Republic—not the Roman Empire—and that his actions helped pave the way for the Empire under his adopted heir Octavian (Augustus). This is historically correct: Caesar was assassinated in 44 BC, and the Roman Empire is conventionally dated from 27 BC.

Both validators failed because the model’s response directly contradicts the expected output, even though the model is right. One of the three runs passed (judge score 8/10, acknowledging the historical nuance); the other two scored 7 and 2, leaving the test with only 1 of 3 passing runs—one short of the majority threshold. The framework behaves correctly here: it surfaces the disagreement and records the full judge reasoning in the JSON report so a developer can inspect the case. Correcting the expected output to “The Roman Republic” would bring the OpenAI pass rate to 100%.

TEST RESULTS — EMBEDDING SCORE · JUDGE SCORE · PER RUN

Test ID	Category	Embedding score / run			Judge score / run			Runs passed	Status
		R1	R2	R3	R1	R2	R3		
history_006	history	0.53	0.54	0.56	8/10	7/10	2/10	1/3	fail

Fig. 6: OpenAI failure — model answer historically more accurate than expected output

G. Ollama Failure Analysis

The Ollama run recorded 9 failures out of 50 cases (82% pass rate). Three factors contribute to this result. First, hardware constraints on the evaluation machine required using llama3.2:3b. Larger models were not feasible because their inference time regularly caused HTTP requests to exceed the configured 100-second timeout, resulting in errors of the form: “The request was canceled due to the configured HttpClient.Timeout of 100 seconds elapsing.” Second, the dataset was limited to 50 cases for the same reason. Third, and most directly visible in the JSON report, llama3.2:3b exhibits systematic judge miscalibration.

The clearest example is test case math_001: prompt “What is 2 + 2?”, expected output “4”, model response “2 + 2 = 4.” The judge’s own chain-of-thought reasoning states: “The actual answer correctly addresses the query. It captures the same meaning as the expected answer. There are no apparent factual errors or key omissions. The actual answer is semantically identical to the expected answer.” Despite this reasoning, the judge assigned a score of 1 out of 10 on all three runs. The embedding score of 0.77 was also below the 0.85 threshold, so both validators failed and the test was marked as failed. This case demonstrates that llama3.2:3b cannot reliably map its own chain-of-thought reasoning to a numeric score.

The Ollama run produced a higher average embedding score (0.67) than the OpenAI run (0.52). This is because nomic-embed-text’s 768-dimensional vectors cluster common factual expressions more tightly than text-embedding-3-small’s 1,536-dimensional space, producing higher raw similarity scores. Despite this advantage, 9 test cases still failed due to judge miscalibration.

Test ID	Category	Embedding score / run			Judge score / run			Runs passed	Status
		R1	R2	R3	R1	R2	R3		
math_023	math	0.60	0.60	0.60	10/10	10/10	10/10	3/3	pass
math_001	math	0.77	0.77	0.77	1/10	1/10	1/10	0/3	fail
reasoning_016	reasoning	0.64	0.64	0.64	2/10	2/10	2/10	0/3	fail
reasoning_004	reasoning	0.56	0.56	0.56	1/10	1/10	1/10	0/3	fail
history_002	history	0.59	0.59	0.59	1/10	1/10	1/10	0/3	fail
history_012	history	0.68	0.68	0.68	7/10	7/10	7/10	0/3	fail
definition_003	definitions	0.67	0.67	0.67	6/10	6/10	6/10	0/3	fail
definition_006	definitions	0.80	0.80	0.80	7/10	7/10	7/10	0/3	fail
definition_010	definitions	0.83	0.83	0.83	7/10	7/10	7/10	0/3	fail
science_014	science	0.71	0.71	0.71	1/10	1/10	1/10	0/3	fail
Embedding: ≥ 0.85 pass 0.70-0.84 borderline 0.50-0.69 low < 0.50 fail Judge: ≥ 8 pass 6-7 borderline < 6 fail									

Fig. 7: Ollama miscalibration — judge reasoning contradicts the assigned score

H. Provider Comparison Tool

It is a standalone browser-based tool that accepts two or three report.json files generated by the framework and renders a structured side-by-side comparison. It is the intended workflow for a team running the same test suite against multiple providers and needing to decide which one to use in production.

Overall Summary section displays pass rate, average embedding score, average judge score, and total test count for each provider side by side. For the OpenAI vs Ollama comparison on the 50-case dataset, this panel immediately shows the headline gap shown in Fig 8.

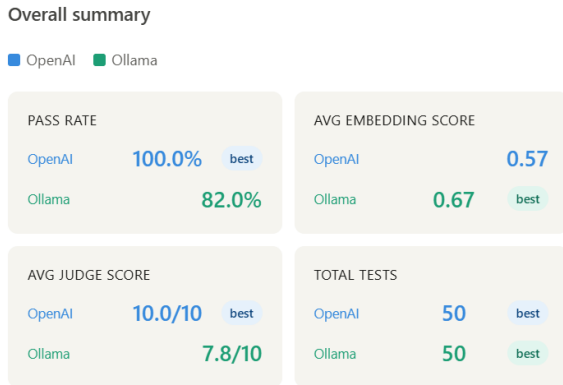


Fig 8: Overall summary comparison between OpenAI and Ollama

Pass Rate by Category panel breaks the comparison down by knowledge domain, rendering a horizontal bar for each provider per category. In the OpenAI vs Ollama comparison, the gap is consistent across all six categories, confirming that Ollama's lower pass rate is a provider-wide calibration issue rather than a subject-matter weakness.

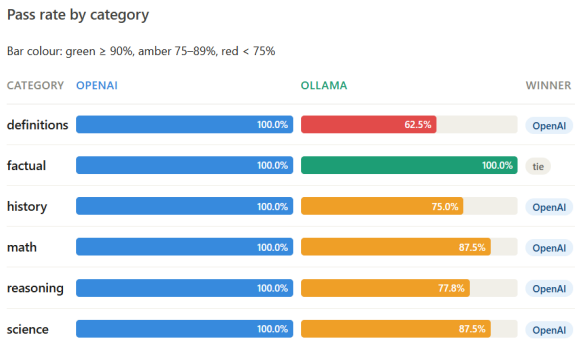


Fig 9: Pass rate by category comparison between OpenAI and Ollama

Score Distributions panel renders two side-by-side histograms: judge score distribution and embedding score distribution, each bucketed into five ranges. A well-calibrated provider produces a right-skewed distribution concentrated at 9–10; a miscalibrated provider produces a bimodal or flat distribution with a significant cluster at 1–3, which represents factually correct answers that the judge scored incorrectly. The embedding histogram confirms that low embedding scores are normal for both providers and validates that the OR logic between the two validators is necessary regardless of which provider is used.

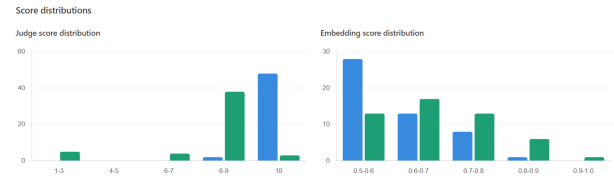


Fig 10: Score distribution comparison between OpenAI and Ollama

The tool also generates an automatic recommendation, naming the provider with the highest overall pass rate and listing the categories in which it leads.

VI. UNIT TESTING

A. Test Strategy and Framework

The test suite comprises 86 unit tests across six test classes and 6 integration tests, all using MSTest (Microsoft.VisualStudio.TestTools.UnitTesting). It has two layers. Unit tests verify the internal logic of each component in isolation: all external dependencies are replaced with mocks or lightweight test doubles, so the full suite runs offline without any API key or running model. Integration tests run the complete pipeline against real OpenAI endpoints to catch issues that mocks cannot simulate, such as unexpected provider response formats or real similarity score behaviour. Integration tests are skipped automatically when the LLM_API_KEY environment variable is absent and can be run explicitly with following command:

```
dotnet test -filter
"TestCategory=Integration"
```

The unit tests are organised into six test classes. CosineSimilarityCalculatorTests contains 16 methods covering geometric correctness, floating-point edge cases, and threshold boundary conditions — no mocking is needed since the class has no external dependencies.

EmbeddingValidatorTests contains 6 methods using Moq to mock IEmbeddingGenerator and ISimilarityCalculator, verifying threshold pass/fail logic, early exit for empty responses, and safe handling of API exceptions.

LLMJudgeValidatorTests contains 8 methods using a hand-written FakeChatClient test double to verify score parsing across seven real judge response formats and threshold logic.

JsonTestLoaderTests contains 14 methods reading and writing real temporary files to verify correct field mapping, optional field defaults, and fail-fast behaviour for malformed or semantically invalid inputs.

TestRunnerTests contains 10 methods verifying the OR pass logic, majority-vote decision (2-of-3), score aggregation, and graceful handling of per-run API exceptions.

ReportGeneratorTests contains 9 methods writing to temporary directories to verify that all four output files are created with correct content.

Every test method carries an XML <summary> comment explaining not just what is being tested but why the assertion matters.

B. Test Double: FakeChatClient

IChatClient from Microsoft.Extensions.AI is used by LLMJudgeValidator and TestRunner to send prompts to any LLM provider. Rather than using Moq — which can interfere with extension methods layered on top of the interface — both test classes use a hand-written FakeChatClient. It manages a queue of pre-staged responses: each call to GetResponseAsync dequeues the next item, returning it as a ChatResponse or throwing it as an exception to simulate API failures. If the queue is exhausted unexpectedly, it throws InvalidOperationException with a clear message so tests fail with a meaningful explanation rather than a silent null result.

C. Integration Tests

The six integration tests run against real OpenAI endpoints and verify behaviour that unit tests cannot cover. Three EmbeddingValidator tests confirm that identical strings produce a cosine similarity of at least 0.95, that a correct full-sentence answer passes the threshold against a short, expected output, and that clearly unrelated strings fall below it. Two LLMJudgeValidator tests verify that a factually correct answer scores at least 7/10 and that a factually wrong answer fails the judge threshold. One end-to-end TestRunner test runs a single test case through the full pipeline and verifies the result contains the correct TestId, TotalRunsCount, and at least one non-zero validator score. Fig. 11 and Fig. 12 show the unit and integration test run outputs.

```

User@DESKTOP-G9B086V MINGW64 /g/FUAS/Rosi/llm-semantic-evaluator/LLMSemanticEvaluatorTests (main)
$ dotnet test
Restore complete (0.5s)
LLMSemanticEvaluator net10.0 succeeded (0.3s) - G:\FUAS\Rosi\llm-semantic-evaluator\LLMSemanticEvaluator\bin\Debug\net10.0\LLMSemanticEvaluator.dll
LLMSemanticEvaluatorTests net10.0 succeeded (0.2s) - bin\Debug\net10.0\LLMSemanticEvaluatorTests.dll
LLMSemanticEvaluatorTests test net10.0 succeeded (3.8s)

Test summary: total: 92, failed: 0, succeeded: 86, skipped: 6, duration: 3.7s
Build succeeded in 5.4s

```

Fig. 11: Unit test run — 86 tests pass, 6 integration tests skipped (no API key)

```

User@DESKTOP-G9B086V MINGW64 /g/FUAS/Rosi/llm-semantic-evaluator/LLMSemanticEvaluatorTests (main)
$ dotnet test --filter "TestCategory=Integration"
Restore complete (0.6s)
LLMSemanticEvaluator net10.0 succeeded (0.2s) - G:\FUAS\Rosi\llm-semantic-evaluator\LLMSemanticEvaluator\bin\Debug\net10.0\LLMSemanticEvaluator.dll
LLMSemanticEvaluatorTests net10.0 succeeded (0.2s) - bin\Debug\net10.0\LLMSemanticEvaluatorTests.dll
LLMSemanticEvaluatorTests test net10.0 succeeded (19.3s)

Test summary: total: 6, failed: 0, succeeded: 6, skipped: 0, duration: 19.3s
Build succeeded in 21.0s

```

Fig. 12: Integration test run — all 6 tests pass against real OpenAI endpoints

VII. DISCUSSION

The central design decision—using Microsoft.Extensions.AI’s IChatClient and IEmbeddingGenerator interfaces rather than writing custom HTTP clients—proved its value across the three supported providers. Changing from OpenAI requires only a Provider

field change in appsettings.json; no code changes are needed anywhere because all consumers depend on the framework interfaces. This is exactly the engineering benefit that justifies adopting a framework rather than implementing custom HTTP wrappers: the framework delivers correct, versioned, and maintained client implementations that the application does not need to build or support over time. The same principle applies to Microsoft.Extensions.Hosting, IOptions<T>, and ILogger<T>: each eliminates a category of boilerplate that would otherwise need to be implemented, tested, and maintained.

The OR combination of embedding and judge validation is confirmed as necessary by the empirical results. With an average embedding score of 0.52 across the OpenAI run, only 6 of 414 individual runs (138 tests × 3 runs) reached the 0.85 threshold. A pure embedding approach would therefore fail roughly 98.5% of runs even though the model’s answers are correct. The structural reason is that the embedding threshold is calibrated for sentence-to-sentence comparisons: when expected outputs are short phrases like “Paris” or “4” and model responses are full sentences, cosine similarity is geometrically low even for semantically identical content. The judge compensates in all but one case, and that one case is a dataset quality issue rather than a model error. The OR design is therefore not a workaround but a structurally necessary choice.

The most significant finding from the Ollama run is that model size and hardware capacity are critical factors for the judge role. The llama3.2:3b model produced a bimodal score distribution where the judge’s own reasoning frequently contradicted the assigned score — the math_001 case being the most extreme example. This is a known limitation of small language models for instruction-following tasks requiring structured output. The framework correctly identifies and surfaces this failure mode through the score distribution chart and per-run reasoning text. For Ollama deployments the recommended minimum is a model of at least 7–8 billion parameters for the judge role, or configuring the judge to use an OpenAI API call while keeping the tested model on Ollama — a mixed configuration supported by the existing codebase since Provider and EmbeddingProvider are independent settings.

Future work could improve both speed and evaluation quality. Independent API calls could be run in parallel to reduce execution time. The embedding validator could also use adaptive similarity thresholds instead of one fixed threshold for all test cases, especially when comparing short expected outputs with longer correct responses. In addition, testing the framework with more providers, larger local models, and broader datasets would help evaluate its generalisability.

VIII. CONCLUSION

This paper presented the LLM Semantic Evaluator, a .NET/C# framework for automated semantic validation of LLM outputs. The framework addresses a practical software engineering problem—automated regression testing for LLM-powered applications—by replacing lexical string matching with two independent semantic validators combined via logical OR and majority-vote aggregation across three repeated runs.

The implementation follows the software engineering principles of the module. Configuration is loaded from `appsettings.json` and bound to a typed class via `IOptions<T>`. All services are registered in the dependency injection container and receive their dependencies through constructor injection. The `Microsoft.Extensions.AI` framework's `IChatClient` and `IEmbeddingGenerator` interfaces decouple all consumers from any specific provider. Switching providers requires only a configuration change. The LLM judge component implements the G-Eval chain-of-thought approach: the judge is required to reason step by step before assigning a score, and the reasoning text is preserved in all reports for auditability.

Evaluation on 138 test cases demonstrated a 99.3% pass rate with OpenAI gpt-5-mini. The one failure is attributable to a test dataset quality issue where the reference expected output was less accurate than the model's response. The 82% Ollama pass rate on a 50-case subset is explained by hardware constraints that required using llama3.2:3b and a reduced dataset due to HTTP timeouts, combined with systematic judge miscalibration in that model. The framework makes both failure modes immediately diagnosable through its structured JSON report. It provides a repeatable, provider-agnostic foundation for LLM regression testing that can be extended to new providers without modifying any existing code.

REFERENCES

- [1] H. Cao, "Recent advances in text embedding: A comprehensive review of top-performing methods on the MTEB benchmark," arXiv preprint arXiv:2406.01607, 2024.
- [2] T. Mikolov et al., "Distributed representations of words and phrases and their compositionality," Advances in Neural Information Processing Systems (NeurIPS), 2013.
- [3] OpenAI, "Text Embedding Models," OpenAI API Documentation, 2024. Available: <https://platform.openai.com/docs/guides/embeddings>.
- [4] N. Muraleedharan, "ML 24/25-09 Semantic Similarity Analysis of Textual Data," Frankfurt University of Applied Sciences, Course Paper, 2025.
- [5] Y. Liu et al., "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment," arXiv preprint arXiv:2303.16634, 2023.
- [6] Ollama, "Get up and running with local language models," 2024. Available: <https://ollama.com>.